CIPHER SOLVERS

TruthSocialist Implementation Report

220659A VARUN.P

Executive Summary

Cyber Cipher is implemented as a synthetic-data MVP for anonymous, eligibility-gated complaints. The implementation separates identified enrollment from anonymous complaint intake, uses RSA blind signatures and Groth16 membership proofs to authorize submissions, encrypts complaint content for the assigned handler, returns a signed receipt, and anchors acceptance and later public actions in an append-only transparency log.

The codebase is deployable as a containerized SaaS or PaaS workload with tenant-fixed configuration and separate PostgreSQL routes for the identity, complaint, handler, and public zones. The browser client demonstrates submission, receipt checking, handler decisions, community voting, and auditor comparison. The five captures in this report are real screens served by the local MVP.

1 Implementation Scope and Status

The TypeScript monorepo separates cryptography, membership, complaint storage, transparency, handler workflow, community review, HTTP services, and the deployable runtime. The prototype accepts synthetic data only.

Area	Implemented capability	MVP evidence
Identity	Synthetic enrollment, depth-16 Poseidon membership, signed checkpoints, device revocation and recovery	Membership and PostgreSQL tests
Entitlement	RSA-3072 RSABSSA-SHA384-PSS-Randomized blind issuance per matter version	RFC 9474 issuance tests
Private proof	Groth16 over BN254 with Poseidon and fixed public-signal order	Circuit compilation and negative witness tests
Complaint	Client encryption, 60-second proof lease, atomic token/nullifier spend, signed receipt	HTTP, store, replay and rollback tests
Transparency	Signed receipt, Merkle proofs, three witnesses and 2-of-3 finality	Receipt and fork tests
Handling and audit	Assigned access, HPKE unwrap, redaction, voting and auditor freeze	Authorization and conflict tests
Deployment	Tenant-fixed runtime and four trust-zone database routes	Compose and runtime tests

Data restriction. The prototype accepts only synthetic identities and complaints. The runtime rejects any data classification other than synthetic, and the development documentation prohibits real NICs and real complaints.

2 Architecture and Deployment

2.1 SaaS and PaaS profile

The prototype runs one stack per tenant and one database route per trust zone. Docker Compose waits for all four databases before starting the non-root, read-only runtime. A production PaaS deployment can use managed PostgreSQL and secret-manager references instead.

Deployment control	Implementation
Tenant binding	MVP_TENANT_ID and MVP_TENANT_SLUG are loaded at startup; requests cannot select another tenant
Database isolation	Distinct URLs for identity, complaint, handler and public zones are mandatory
Network exposure	Only the runtime publishes a loopback port; database networks are internal
Container hardening	Non-root Node user, read-only filesystem, no-new-privileges and all capabilities dropped
Transport and data	Production requires HTTPS and database TLS; the MVP permits synthetic data only

3 End to End Workflow

1. The institution verifies eligibility. The Identity Authority adds a membership leaf and blind-signs a matter entitlement without learning its final serial.
2. The client encrypts the complaint, obtains a 60-second proof lease, and proves current membership. Intake atomically spends the entitlement and nullifier, stores ciphertext metadata, and returns a signed receipt.
3. An assigned WebAuthn-authenticated handler unwraps only the case key and records signed access, lifecycle events, and a finding. An independent reviewer approves any redacted public derivative.
4. Eligible members vote anonymously once. The auditor compares the vote with the handler finding and may freeze only the public derivative when outcomes conflict.

4 Security and Cryptographic Implementation

Purpose	Selected mechanism	Implemented rule
Eligibility entitlement	RSA-3072 RSABSSA-SHA384-PSS-Randomized	One issuance fact per enrollment and matter version; client verifies final token
Private membership	Groth16 over BN254 with Poseidon	Current root only; complaint and vote circuits use frozen public-signal order
Complaint encryption	AES-256-GCM	Client encrypts content; routing and commitment fields are authenticated data
Handler key routing	X25519 HPKE with HKDF-SHA256 and AES-256-GCM	Only a 32-byte data-encryption key is wrapped to the assigned handler key
Signatures	Ed25519	Separate keys sign checkpoints, leases, receipts, logs, witnesses and auditor actions
Recovery	128-bit random seed and ID, Ed25519 recovery key	Five-minute challenge and mandatory rotation after successful recovery
Canonical data	Deterministic CBOR and SHA-256	Strict decoder rejects unknown, duplicate and non-canonical values
Public log	RFC 6962-style SHA-256 Merkle tree	Three witnesses; two signatures required for finality

4.1 Recovery and privacy boundary

A pre-shared 128-bit recovery seed signs a five-minute challenge. Success revokes the old leaf, allocates a never-reused index, activates the new device key, and rotates the seed. The Identity Authority sees enrollment and issuance facts but not the final blind token; complaint intake sees a proof and ciphertext but no identity reference. Timing and infrastructure correlation remain outside this MVP guarantee.

5 Complaint Logging Checking and Approval

5.1 Complaint logging

The service stores encrypted bytes in a content-addressed object store and sends only their URI, hash, size, nonce, wrapped data key, and handler key ID to PostgreSQL. One transaction records the complaint, spent serial, used nullifier, log outbox event, signed receipt, consumed proof session, and idempotency result.

5.2 Handler checking

A P-256 WebAuthn-authenticated handler can open an assigned case; a supervisor can open cases in the same organization. Every allowed or denied read creates a signed access event. Valid state transitions and expected row versions prevent unauthorized or stale updates.

5.3 Publication approval

Public release requires complainant opt-in, automated PII checks, and approval by a reviewer other than the drafting handler. Approval opens a seven-day voting window; rejection keeps the derivative private.

5.4 Community and auditor decision

The vote circuit proves current membership and derives a complaint-scoped nullifier, preventing a second vote even after device recovery. Results need ten votes and a 60 percent principal choice. The auditor signs the comparison; a conflict may freeze the public derivative but never the private complaint or handler history.

6 Database Implementation

Package-owned PostgreSQL migrations enforce tenant context, cryptographic lengths, valid states and windows, serial and nullifier uniqueness, and one-to-one receipts.

Schema	Principal tables	Integrity purpose
ida	enrollment, membership_leaf, membership_checkpoint, recovery_challenge, matter_issuance	Unique synthetic identity, monotonic leaf allocation, signed root history, one-time recovery and issuance
complaint_store	matter, proof_session, spent_entitlement, used_nullifier, complaint_record, receipt_record, log_outbox	Atomic anonymous acceptance and replay prevention
handler_workflow	staff_credential, case_record, case_event, case_access_event, mailbox, mailbox_message	Assignment-only access and append-only accountability
transparency_log	log_leaf, tree_head, witness_signature	Public inclusion, consistency and quorum evidence
community	redaction, vote, auditor_assessment, transparency_outbox	Opt-in publication, one vote per nullifier and immutable auditor evidence

6.1 Replay prevention schema

Listing 1 Database uniqueness prevents entitlement and proof replay

```
CREATE TABLE complaint_store.spent_entitlement (
  matter_key_id bytea NOT NULL CHECK (octet_length(matter_key_id) = 32),
  serial bytea NOT NULL CHECK (octet_length(serial) = 16),
  spent_at timestamptz NOT NULL,
  PRIMARY KEY (matter_key_id, serial)
);

CREATE TABLE complaint_store.used_nullifier (
  matter_id uuid NOT NULL,
  matter_version bigint NOT NULL,
  nullifier numeric(78,0) NOT NULL,
  used_at timestamptz NOT NULL,
  PRIMARY KEY (matter_id, matter_version, nullifier)
);
```

Handler events, access records, mailbox messages, votes, and auditor assessments are append-only. Database permissions and triggers reject mutation; case and mailbox hash chains expose reordering or deletion.

7 API and Runtime Implementation

Boundary	Method and route	Function
Identity	POST /ida/v1/enrollments	Authenticated synthetic enrollment
Identity	POST /ida/v1/matters/{id}/{version}/blind-issuance	Blind entitlement issuance
Identity	POST /ida/v1/recovery/*	Five-minute challenge and atomic device replacement
Public	GET /public/v1/membership/checkpoints/current	Current signed membership root
Complaint	POST /complaints/v1/proof-sessions	Signed 60-second current-root lease
Complaint	POST /complaints/v1/complaints	Anonymous encrypted complaint submission
Runtime	GET /health/* and /public/v1/config	Safe readiness and public protocol configuration

Security-sensitive bodies use deterministic CBOR; UUIDv4 idempotency keys protect mutations. Anonymous complaint responses use no-store. JSON is limited to health and public configuration.

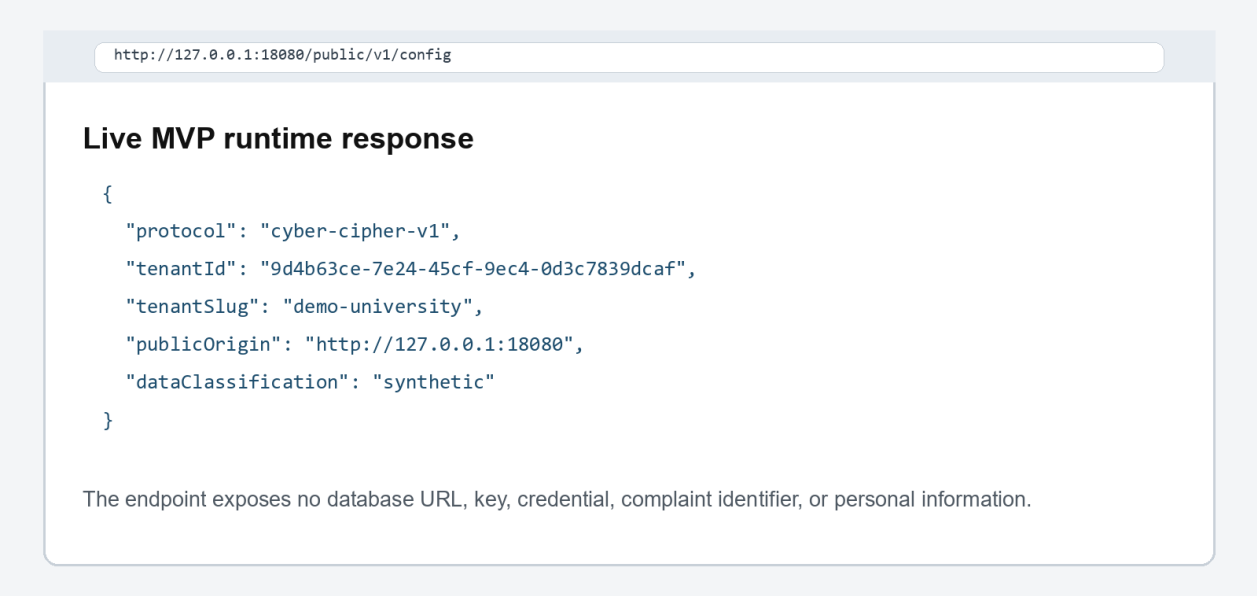


Figure 1 Live public configuration response from the MVP runtime

8 Implemented Demonstration Interface

The browser client uses the MVP HTTP routes to demonstrate the synthetic journey. Presentation state resets with the runtime; cryptographic packages and database adapters are tested separately.

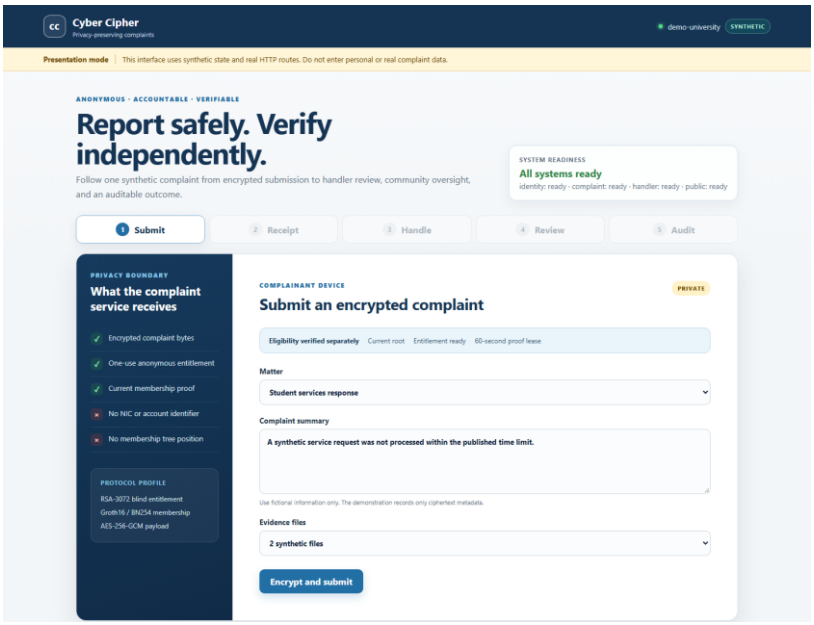


Figure 2 Live complaint submission screen

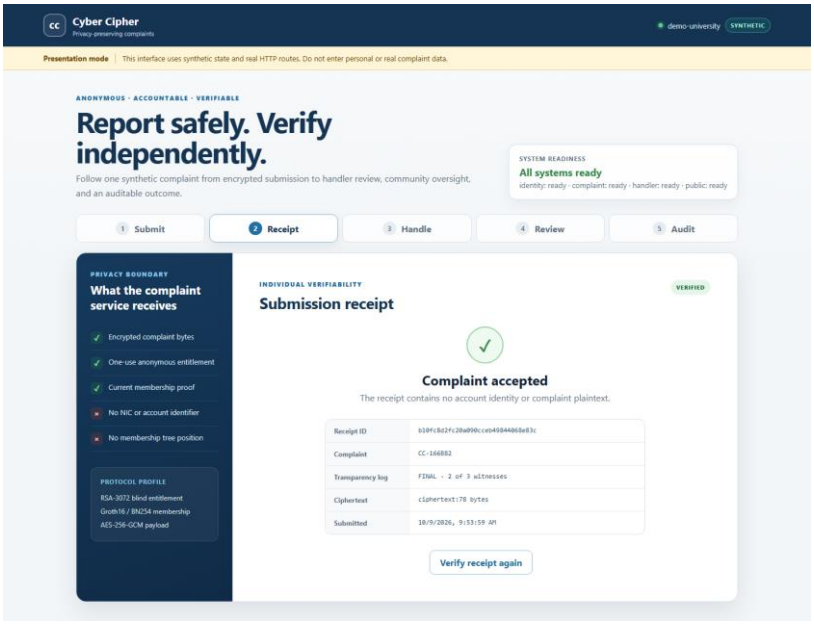


Figure 3 Live signed-receipt and public-log status screen

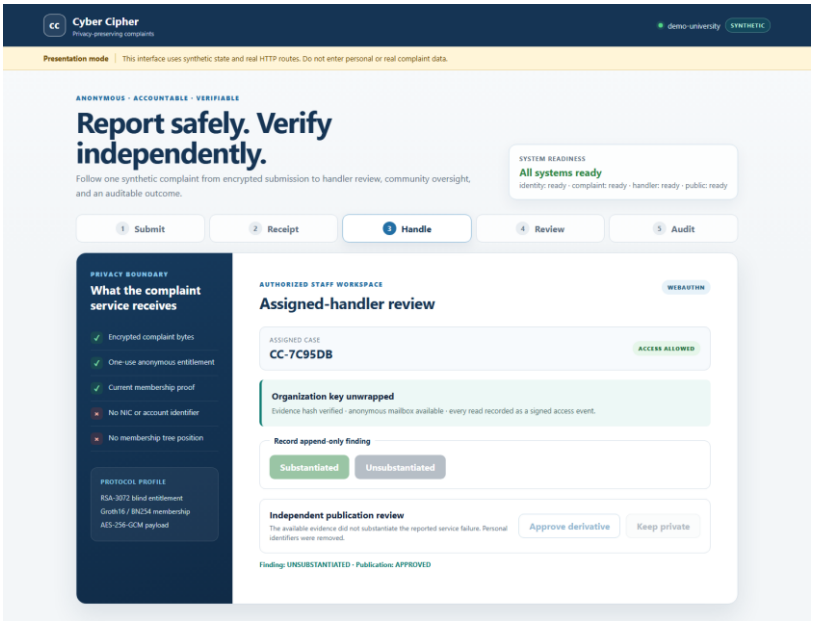


Figure 4 Live assigned-handler review and decision screen

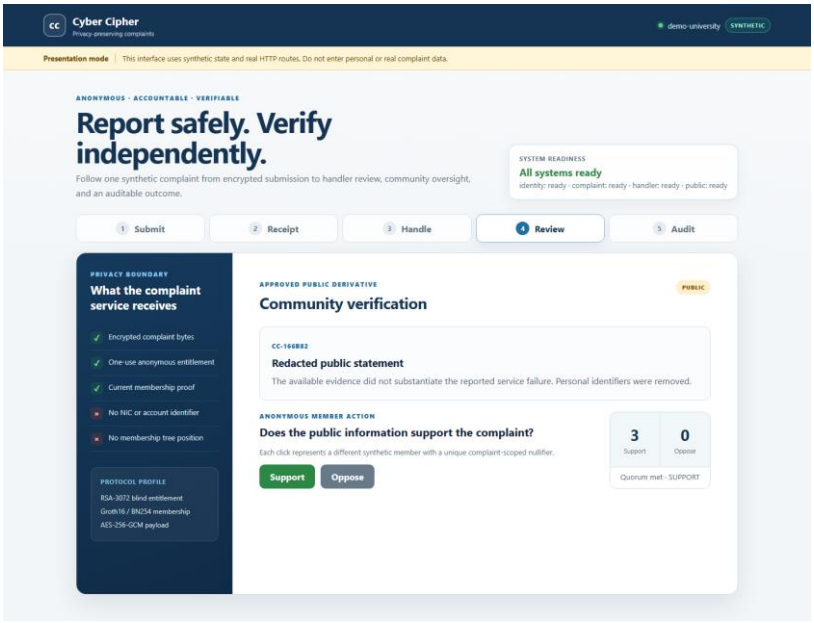


Figure 5 Live community voting screen

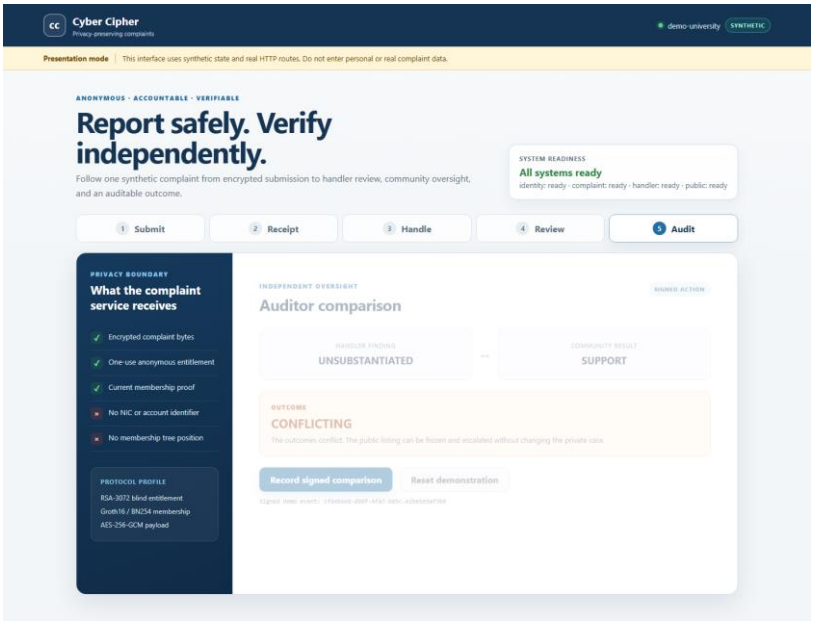


Figure 6 Live auditor comparison and action detail

User	Primary action	Privacy or authorization control
Complainant	Choose matter, prepare evidence, submit and retain receipt	Encryption and proof generation remain on the device
Handler	Open assigned cases, request information and record finding	WebAuthn, organization and assignment checks
Reviewer	Approve or reject redaction	Independent from drafting handler
Community member	Vote on approved derivative	ZK proof and scoped nullifier
Auditor or public	Compare outcome or verify receipt	Signed evidence; no complainant identity

9 Runtime Evidence and Test Results

9.1 Guided demonstration

The captured protocol run completed in 158.0 seconds with 79 passing checks from 83 discovered; four local PostgreSQL checks were skipped because Docker was unavailable. CI for commit 8c3cc95 passed with PostgreSQL. The browser runtime then passed 9 of 9 focused checks, including security headers and the complaint-to-auditor journey.

9.2 Acceptance coverage

Test area	Observed result
Encoding, crypto and membership	31 of 31 guided checks passed
RSA entitlement and Groth16	23 local checks passed; database case exercised in CI
Complaint, transparency and receipt	10 local checks passed; database case exercised in CI
Handler, community and auditor	8 local checks passed; database cases exercised in CI
SaaS runtime and browser client	9 of 9 focused checks passed

CI evidence. GitHub Actions run 37583750614 for commit 8c3cc95 validated deployment policy, TypeScript, circuit compilation, package tests, and PostgreSQL integration.

9.3 Tested failure cases

- Malformed encoding, signatures, leases, challenges and ZK proofs are rejected.
- Serial, complaint-nullifier, vote-nullifier and conflicting-idempotency replay is rejected.
- Unauthorized access, invalid transitions, PII, missing opt-in and self-review are blocked.
- One witness cannot finalize a head; a same-size fork produces gossip evidence.

10 Running the MVP

10.1 Install and test

Listing 2 Local build, acceptance and guided demonstration

```
cd C:\Users\varun\Desktop\Academics\S7\INS\truthsocialists
pnpm install --frozen-lockfile
pnpm acceptance:mvp
pnpm demo:guided
```

10.2 Run with Docker Compose

1. Start Docker Desktop.
2. Copy infra/compose/.env.mvp.example to infra/compose/.env.mvp and replace the password with a random URL-safe local value.
3. Run docker compose --env-file infra/compose/.env.mvp -f infra/compose/docker-compose.mvp.yml up --build -d.
4. Open http://127.0.0.1:8080/ and submit the prefilled synthetic complaint.
5. Use the receipt, handler, community and auditor tabs to demonstrate the workflow. Use synthetic data only, then stop the same Compose profile with down.

11 Limitations and Next Work

The MVP is not approved for real users. The browser uses in-memory synthetic presentation state and does not yet perform proving, blind-token unblinding, evidence encryption, WebAuthn, or offline verification on the client. A pilot requires independent cryptographic and penetration testing, privacy and legal review, accessibility testing, production key and object storage, TLS, backups, rate limiting, incident response, and named independent witnesses.

Status	Meaning
Completed for experimental MVP	Protocol packages, service adapters, database migrations, circuits, Compose profile, automated acceptance tests and synthetic browser workflow
Demonstration-only	Browser presentation state and simulated role transitions; state resets on runtime restart
Required before pilot	External security, privacy, legal, accessibility and operational approval
Prohibited now	Real NICs, real complaints or claims of production anonymity